

Prompt

Prompt 1: The Core Foundation (Base Logic)

"Write a Python script that acts as a grade auditor. It should calculate a final class grade using a 50/50 weight split: 50% from quizzes and 50% from a final exam. Set up a standard grading scale where A is 90-100, B is 80-89.9, C is 70-79.9, D is 60-69.9, and F is anything lower. Use a command-line interface to collect the grades."

Rules

- **Quizzes:** 50% of the grade (Lowest 1 quiz is dropped).
- **Final Exam:** 50% of the grade.
- **The Grading Scale (Ranges)**

Letter Grade	Percentage Range	What it Means
A	90% to 100%	Excellent
B	80% to 89.9%	Good
C	70% to 79.9%	Satisfactory
D	60% to 69.9%	Passing
F	Below 60%	Failing

Prompt 2: Smart Parsing & Flexibility (Regex Input)

"Refine the script so that users don't have to input percentages manually. I want them to be able to type raw marks as fractions (like 15/20 or 42/50). Use regular expressions so they can paste a whole string of quiz scores separated by commas or spaces at once (e.g., 15/20, 18/20, 9/10) and have the script automatically extract and calculate them."

Prompt 3: Adding Class Policy (Drop Lowest Quiz)

"Add a 'drop the lowest quiz' rule to the calculation. If the user enters 2 or more quizzes, find the quiz that has the lowest *percentage* score, remove it from the active calculation, and flag it as 'dropped' so the user knows it didn't count against them. Make sure it drops based on the percentage performance, not just raw point values."

Prompt 4: Predictive Analytics (Target Forecast Mode)

"Make the final exam entry conditional. When asking for the final exam score, let the user press Enter to leave it blank if they haven't taken it yet. If left blank, prompt them for their *desired* target letter grade (A, B, C, or D). Calculate the exact score they need on the final exam to hit that target. If it's impossible, tell them. Also, generate a complete matrix showing what they would need on the final to hit *every* possible passing grade bracket."

Prompt 5: UI & Visual Polish (HTML Report Generation)

"Instead of just printing the final summary to the terminal, let's turn this into a visual report. Map each grade to a hex color code (like green for A, red for F). Use Python to dynamically generate a clean, modern HTML file containing styled CSS cards, metric grids, status badges for active/dropped quizzes, and a clean target matrix table. Use the `webbrowser` module to automatically save the file locally and open it in the user's browser when the script finishes. Wrap everything in a master try/except block so it handles bad user input gracefully without crashing."

Logic

1. Setup and Helper Functions

At the beginning, the script defines a standard grading scale (\$A\$ to \$F\$) mapped to colors and percentages. It then establishes two key helper functions:

- `get_letter_grade(percentage)`: Loops through the grading scale to match a percentage score to its corresponding letter grade, description, and color palette.
- `parse_fraction(text)`: Uses Regular Expressions (Regex) to parse text inputs like "15/20" or "45 / 50". It extracts the two numbers, converts them to floats, and returns them as (earned, total). If only one number is provided, it assumes it's a score out of 100.

2. Interactive CLI Inputs

The script prompts the user through a terminal interface to gather data:

- **Quizzes:** It asks for quiz scores in an Earned/Total format (e.g., 12/15, 18/20). It processes this input using Regex to find all matching fractional pairs and stores them in a list of dictionaries, calculating individual percentages along the way.
- **Final Exam:** It asks for the total possible marks for the final exam. It then asks the user if they have already taken the exam.
 - If they **have** taken it, they input their score.
 - If they **haven't** taken it, they press ENTER to leave it blank, triggering the "Target Forecast" mode.

3. The "Drop the Lowest Quiz" Logic

Before calculating the final grades, the script clones the quiz list to keep a raw history, then performs the following steps:

1. **Check count:** If the user entered two or more quizzes, it evaluates them.
2. **Find the minimum:** It uses Python's min() function paired with a custom key (key=lambda x: x["pct"]) to find the quiz with the lowest performance *percentage* (not raw points).
3. **Drop it:** It removes that lowest quiz from the active calculations list.
4. **Aggregate:** It sums up all the earned and total marks of the remaining *active* quizzes to calculate the quiz_average_pct.

4. Dual-Mode Evaluation Logic

The core branching logic depends entirely on whether a final exam score was provided:

Mode A: If the Final Exam score is provided

The script calculates the total grade based on a strict 50/50 split:

$$\\$\\text{Overall Score} = (\\text{Quiz Average } \\% \\times 0.5) + (\\text{Final Exam } \\% \\times 0.5)\\$\\$$$

It determines the letter grade using this score and builds a HTML snippet summarizing the student's current fixed standing.

Mode B: If the Final Exam score is blank (Target Forecast)

The script asks the user what grade they want to achieve (\$A\$, \$B\$, \$C\$, or \$D\$). It looks up the minimum percentage required for that grade and runs an algebraic forecasting calculation to see what score is needed on the final:

$$\begin{aligned} \text{Final \% Needed} &= \frac{\text{Target \%} - (\text{Quiz Average \%} \times 0.5)}{0.5} \\ \text{Marks Needed} &= \frac{\text{Final \% Needed}}{100} \times \text{Final Total Marks} \end{aligned}$$

It then checks for three logical scenarios:

- **Marks Needed ≤ 0 :** The student has already secured the grade even if they get a 0% on the final.
- **Marks Needed $> \text{Total Exam Marks}$:** Even a perfect score on the final exam won't save them; the target is mathematically impossible based on past quiz performance.
- **In-between:** It calculates the exact threshold score they must achieve on the exam.

It then dynamically builds a full grid matrix showing the exact scores required for *all* passing letter grades.

5. Report Generation and Display

Finally, the script compiles all parsed data, active/dropped quiz lists, and calculated statistics into a heavily styled HTML string template.

- It writes this text block to a file named `grade_report.html` in the script's directory.
- It utilizes Python's `webbrowser.open()` module to automatically trigger the host computer's default web browser, displaying the polished visual report immediately.
- The entire process is wrapped in a `try/except` block to ensure that if a user types something completely unparsable, the program catches the error gracefully, prints a warning message, and waits for ENTER instead of instantly crashing out of the terminal.